6.2.1. Variables, assignment, and side effects

Variable assignment is at odds with functional programming. Sure, assignment looks harmless enough,

```
state = on
state = off
```

until you discover what happens when you try to glue together functions that assign variables. Consider the `numberSold` function that evaluates to the number of units sold when invoked with a given sale price:

```
numberSold = (salePrice) ->
  50 + 20/10 * (200 - salePrice)
```

Again, the formula that Agtron has worked out for the number of items sold at a given sale price

Compare it to a `calculateNumberSold` function that returns the value *and* sets a variable:

```
numberSold = 0
calculateNumberSold = (salePrice) ->
  numberSold = 50 + 20/10 * (200 - salePrice)
```

This assignment is a side effect.

These functions evaluate to the same value, but the `calculateNumberSold` function assigns a value to an outer variable (a variable that's not contained entirely within the function):

```
calculateNumberSold 220
# 10
```

Consider now the `calculateRevenue` function that uses the `numberSold` variable:

```
calculateRevenue = (salePrice) ->
  numberSold * salePrice
```

Suppose you want to show Agtron a graph of the `revenue` for different cameras at different prices. To do this, you start by calculating the revenue for prices between 140 and 145 in a comprehension: